# Business Email Compromise Response

An outsider took over a staff email account, hid client replies with an inbox rule, and sent fake payment instructions. In one evening I found out how they got in and handed IT a report it could verify.

| ROLE | ENVIRONMENT | ACCESS | TIMEFRAME |
|---|---|---|---|
| Independent IT support | Microsoft 365, Windows | Staff-level only, no admin | One evening |

**700+**
attacker sign-in events traced to one stolen session

**12,000+**
sign-in log rows analysed

**65+**
evidence files, each hashed

**3**
audiences: owner, staff, IT

## THE SITUATION

A small professional-services firm found fake payment instructions going out from its own mailboxes. Its IT support and owner were unavailable after hours, and sign-in logs were expiring daily.

## MY ROLE

I was engaged to investigate remotely on the affected staff member's computer and account. I contained the account, collected and analysed the evidence, and wrote the report and guides.

## INSIDE

Constraints I worked under

What I did, step by step

What the evidence showed

Decisions and self-corrections

Deliverables, tools and lessons

## WHAT IT SHOWS

- Incident response under pressure: contained first, preserved evidence before every change
- Log analysis: Microsoft Entra sign-in exports analysed in Python and pandas
- Evidence handling: SHA-256 chain of custody, every finding traced to source rows
- Clear writing for non-technical owners, with a technical appendix for IT

## Constraints

- **No admin rights.** The account had no tenant admin role, so the audit log, message trace and other users' sign-in logs were out of reach.
- **Expiring evidence.** Sign-in logs were kept for only 7 days, so evidence disappeared daily.
- **Limited host history.** The computer had been reinstalled 10 days earlier.
- **After hours.** The owner and IT were unavailable, so anything needing their approval became a written recommendation.
- **Cleanup already started.** I had to preserve what remained without destroying more.

## What I did

1. **Triage and containment.** Found a malicious inbox rule (named ".") that deleted and marked as read every message from a client's address and a whole public email domain. I screenshotted it before changing anything, then guided the password reset and sign-out of all sessions.
2. **Runbook review.** Reviewed and rewrote a 45-step compromise response runbook. I moved containment ahead of cleanup, replaced screenshot-only evidence with admin exports, and added the systems an inbox unlocks, such as practice management, accounting and banking.
3. **Endpoint triage.** Wrote read-only PowerShell collection scripts and ran them twice, once as a standard user and once elevated. They covered Defender status and exclusions, persistence (Run keys, scheduled tasks, services, WMI subscriptions), remote-access tools, browser extensions, Outlook add-ins, network settings, root certificates, logon events (4624/4625), new services (7045) and PowerShell script-block logs. I hashed every output with SHA-256.
4. **Cloud log analysis.** Exported the four Entra sign-in files (12,000+ rows) and analysed them in Python and pandas. I grouped them by ASN, IP and session ID, joined the authentication-detail files on Request ID, and paired legitimate and attacker sign-ins by time to find impossible travel.
5. **Email forensics.** Recovered a forwarded copy of the fraud thread from Deleted Items and parsed its headers. The authentication results showed the forward was genuine. `OFR:CustomRules` proved the inbox rule was still active. The Message-IDs mapped each fake email to the firm's own mail servers, which ruled out a look-alike domain.
6. **Chain of custody.** Built an evidence manifest script (a SHA-256 hash for every file, plus a fingerprint for the manifest itself) and a rename script that logs the old name, new name and unchanged hash. Each finding extract carries its source file and line number.
7. **Reporting.** Wrote a 16-page report in two tiers: plain language for the owner and staff, and a technical appendix for IT with indicators of compromise, sign-in tables, header analysis, commands and steps to verify each finding independently. I also wrote a 2-page, 6-step guide for the owner to secure his own account.

## What the evidence showed

| Finding | Evidence |
|---|---|
| All 700+ attacker events over 5 days came from **one stolen session** | Entra sign-in exports |
| **MFA was never prompted.** Every event reused a token that had already passed MFA, consistent with adversary-in-the-middle phishing | Authentication details, joined on Request ID |
| About 10 attacker IP addresses on two data-centre networks, with check-ins roughly every 4 hours, which points to automation | ASN and IP grouping |

| Finding | Evidence |
|---|---|
| **Impossible travel:** legitimate sign-ins from the user's usual location and attacker activity overseas, as little as 2 minutes apart | Time-paired sign-ins |
| Attachments were downloaded through the session, so client attachments had to be treated as exposed | Non-interactive resource field |
| The same malicious rule in two mailboxes, with delegate access suggesting one session reached both | Screenshots, mailbox URLs |
| The fake emails were sent from inside the firm's tenant, not spoofed | Message-ID server mapping |
| The computer was clean: no malware, persistence or remote-access tools, and all logons local | PowerShell triage |

## Decisions and self-corrections

- **Read-only first.** I held off on offline scans, extension removal and rule deletion until each item was recorded. A reboot or quarantine can destroy evidence.
- **Targeted second pass.** My first script run wasn't elevated, so some security data was missing. Instead of rerunning everything, I wrote a short admin follow-up script.
- **Testing my own conclusions.** I flagged a sign-in from a cloud IP address as suspicious, then confirmed it was the firm's own virtual PC and withdrew it. I first concluded that both mailboxes were signed into separately. Mailbox URL evidence showed one session could explain both, so I reframed it as an open question for IT.
- **Checking what looked odd.** Two downloaded installers had identical sizes. Their hashes differed and both carried valid Microsoft signatures, which fits normal Store installer stubs.
- **Honest scope.** The report separates "the evidence shows" from "most likely," states its limits, and tells IT how to verify each finding against the tenant.
- **Racing retention.** I ranked IT's actions by when evidence would expire: sign-in logs (7 days), message trace (10 days), recoverable items (14 days), audit log (about 180 days).

## Deliverables

- A revised 45-step compromise response runbook
- Three PowerShell scripts (triage, admin triage, evidence manifest) and a hash-verified rename script
- 9 finding extracts with a findings index
- A 16-page two-tier incident report and a 2-page account-securing guide for the owner

## Tools

Microsoft Entra admin center, Microsoft 365 My Sign-ins, Outlook on the web, Windows PowerShell 5.1, Microsoft Defender, and Python (pandas). I used an AI assistant (Claude) to speed up parsing and drafting, and checked every conclusion against the raw data.

## Lessons learned

- **MFA is not the finish line.** Push-based MFA was in place and was bypassed. Phishing-resistant passkeys, Conditional Access and shorter sessions close the gap.

- **Seven days of logs is not enough.** On a small tenant, sign-in logs disappear before most people notice an incident.
- **Inbox rules are a warning sign.** One-character rule names with delete and mark-as-read actions are a classic marker, and alerts on them should reach someone who reads them.
- **Evidence first, cleanup second.** Every capture taken before a change made the final report defensible.
- **Write for the reader.** The most useful output wasn't the analysis. It was a 2-page guide a busy owner could follow in about 18 minutes.